

Unit VI – Applications of Data Structures

1. Stacks (Teaching Notes in C)

1.1 Introduction to Stacks

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The element inserted last is removed first.

Examples: Stack of plates, undo operations, browser history.

Basic Operations on Stack

push(), pop(), peek(), isEmpty(), isFull()

C Program – Stack Implementation using Array:

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int val) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else {
        top++;
        stack[top] = val;
    }
}

void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else {
        printf("Popped element: %d\n", stack[top]);
        top--;
    }
}

void display() {
    if (top == -1)
        printf("Stack is Empty\n");
    else {
        printf("Stack elements: ");
        for (int i = top; i >= 0; i--)
            printf("%d ", stack[i]);
        printf("\n");
    }
}

int main() {
    push(10);
    push(20);
```

```

    push(30);
    display();
    pop();
    display();
    return 0;
}

```

Output:

```

Stack elements: 30 20 10
Popped element: 30
Stack elements: 20 10

```

1.2 Conversion of Arithmetic Expressions

Three forms: Infix (A+B), Prefix (+AB), Postfix (AB+)

C Program – Infix to Postfix Conversion:

```

#include <stdio.h>
#include <ctype.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

void infixToPostfix(char *exp) {
    char *e = exp;
    while (*e != '\0') {
        if (isdigit(*e))
            printf("%c", *e);
        else if (*e == '(')
            push(*e);
        else if (*e == ')') {
            while (stack[top] != '(')
                printf("%c", pop());
            pop();
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(*e))
                printf("%c", pop());
            push(*e);
        }
        e++;
    }
    while (top != -1)
        printf("%c", pop());
}

```

```

int main() {
    char exp[] = "A+B*C";
    printf("Infix Expression: %s\n", exp);
    printf("Postfix Expression: ");
    infixToPostfix(exp);
    return 0;
}

```

Output:

Infix Expression: A+B*C
 Postfix Expression: ABC*+

1.3 Applications of Stack

Reversing a String in C

```

#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int main() {
    char str[100];
    printf("Enter a string: ");
    gets(str);
    for (int i = 0; i < strlen(str); i++)
        push(str[i]);

    printf("Reversed string: ");
    while (top != -1)
        printf("%c", pop());
    return 0;
}

```

Parentheses Checker in C

```

#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int isMatching(char a, char b) {
    return (a == '(' && b == ')') ||
           (a == '{' && b == '}') ||
           (a == '[' && b == ']');
}

int isBalanced(char exp[]) {

```

```

    for (int i = 0; i < strlen(exp); i++) {
        if (exp[i] == '(' || exp[i] == '{' || exp[i] == '[')
            push(exp[i]);
        else if (exp[i] == ')' || exp[i] == '}' || exp[i] == ']') {
            if (top == -1)
                return 0;
            if (!isMatching(pop(), exp[i]))
                return 0;
        }
    }
    return (top == -1);
}

int main() {
    char exp[100];
    printf("Enter expression: ");
    gets(exp);

    if (isBalanced(exp))
        printf("Expression is Balanced\n");
    else
        printf("Expression is Not Balanced\n");
    return 0;
}

```